B10 Feature engineering without leakage

Target leakage, contamination and time-safe features.

B10-L1 Target leakage, features that know the answer

For every feature, know when it becomes available. Screen single-feature AUCs, and treat a near-perfect one as a suspect.

```python
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y, X[col])
strength = max(auc, 1 - auc)
```

The one question for every feature	Would I know this value at prediction time?
Single-feature leakage screen	AUC of each feature alone. Flag max(auc, 1 - auc) near 1.
Common leaks	Consequences of the outcome, values measured after the prediction date, labelling proxies, order-encoding IDs.
Feature timeline	A table of when each feature becomes known, reviewed with the data owner.

B10-L2 Fit on training data only

Split first. Fit every learning step on training rows. Apply the fitted parameters to everything else.

```python
params = fit_preprocessing(train)          # learn from train only
train_ready = apply_preprocessing(train, params)
test_ready = apply_preprocessing(test, params)   # reuse, never refit
```

Train/test contamination	A learning step saw test rows: imputation, scaling, encoding, selection.
Selection on all rows, noise data	CV accuracy 0.85. Inside the folds: 0.48. The first number is a leak.
The manual rule	Split first, fit on train, apply the fitted parameters to train and test.
Safe before the split	Steps that learn nothing from the rows: fixed recodes, unit conversions, parsing.

B10-L3 Time-safe features and point-in-time joins

Compute every feature as of the row's date, join with merge_asof, and split by time.

```python
orders = orders.sort_values(["customer", "ordered_at"])
prior = orders.groupby("customer")["spend"].cumsum() - orders["spend"]
```

```python
pd.merge_asof(preds.sort_values("as_of"), scores.sort_values("scored_at"),
              left_on="as_of", right_on="scored_at", by="customer",
              allow_exact_matches=False)
```

Prior-only running total	groupby(key)[col].cumsum() - col, after sorting by key and date.
Point-in-time join in pandas	pd.merge_asof(left, right, left_on, right_on, by=..., allow_exact_matches=False), both sorted.
merge_asof default for equal timestamps	allow_exact_matches=True: same-time rows match. Set False when they would not be known yet.
Split for forward-looking models	By time: train before a cut-off, test after. TimeSeriesSplit for CV.